Beyond vibe coding

Retour à « Resources »

Beyond Vibe Coding

https://vibecoding.holt.courses/

Brian Holt's workshop, free on master.dev. It is a technical course: how to actually build software with an agent sitting next to you. That is adjacent to this course rather than the same thing — we are about *managing* the work, it is about *doing* it — but the two reinforce each other, and the whole workshop is worth your time if you want it. Take it on your own schedule, outside this class.

Two sections earn a place on the reading list.

Section 10 — Putting It All Together

- MCP
- GitHub for Real
- Reading a Codebase
- Project: Stranger's Code

The lesson that matters most here is **Reading a Codebase** and the project attached to it. Every competency in this course eventually lands on the same problem: you are dropped into code you did not write and have to form an accurate picture of it fast — with an agent, without letting the agent's confident summary stand in for your own understanding. That is C3's opening move, and it is what the shared repository will ask of you on 8 September.

Project: Stranger's Code is the part to actually do. You fork a full-stack app you have never seen — Postgres, an Express API, a frontend — and work through it in three passes. First a scavenger hunt: ten questions about the codebase, answered **without the agent**, each answer citing a file and a line number. Then you pick one small feature and build it on a branch, as a pull request. Then you review your own PR against the conventions the codebase already has, rather than the ones you would have chosen.

Those three passes are the same shape as C1 → C2 → C3 in miniature, and the first one is the one students skip. Answering ten questions with your own eyes, with citations, is what makes the difference between knowing a codebase and having been told about it — and on 11 September it is your reading of the shared repository, not the agent's, that gets interviewed.

> **A note on `/teach`.** Matt Pocock's `/teach` skill (`npx skills@latest add mattpocock/skills`) turns an agent into a tutor: it interviews you about your goal and level, then generates lessons and quizzes and keeps a record of what you have covered. It is a good fit for exactly this section — use it on the parts of the stack the scavenger hunt exposes as gaps, when the honest answer to a question was "I found the line but I do not know what it does".
>
> Note what it is and is not. It moves the agent from doing the work to checking whether *you* can do it, which is the right direction here. It is also not a substitute for the scavenger hunt itself: a lesson you were taught is not a codebase you have read, and the oral can tell the two apart.

Section 11 — Debugging

- How to Think About Bugs
- Debugging with an Agent
- Preventing Bugs
- Project: Bug Backlog

This is C3 taught from the other end. **How to Think About Bugs** and **Debugging with an Agent** cover the same discipline the assessment grid rewards: reproduce before you theorise, let the loop reject your hypotheses instead of your reading of the code. **Preventing Bugs** is the "where does the finding belong" question — specification, instruction document, or the code's own structure.

The bug backlog is free exam practice

Project: Bug Backlog hands you a repository seeded with real bugs and an issue tracker full of them:

https://github.com/btholt/family-recipe-box-broken/issues

That is a rehearsal for C3 and for the oral, and it costs you nothing. The shared repository you get on 8 September works the same way: an unfamiliar codebase, a bug you did not write, a fix you have to *verify* rather than believe. Work a few issues from this backlog beforehand and, for each one, produce what C3 asks for — one command that goes red on the bug and green when it is fixed, a stated cause with the evidence that pointed at it, and a regression test that fails on the original code.

> Do that three or four times and the exam is a repeat of something your hands already know. Turn up having never reproduced a bug you did not write, and you will spend the autonomy slot discovering the method instead of applying it.

Modifié le: mercredi 2 septembre 2026, 19:20